Distributed Systems
2026WS

universität innsbruck

703088 (2026WS) - PS Distributed Systems

Distributed and Parallel Systems Group
University of Innsbruck Authors: Marlon Etheredge, Juan Aznar Poveda In case of questions regarding this task, please contact the authors ({marlon.etheredge, juan.aznar-poveda}@uibk.ac.at). Instructors: Juan Aznar Poveda, Zahra Najafabadi Samani, and Marlon Etheredge

EX05 - Fault Tolerance & Coordination

Introduction

In the previous exercises, you explored peer discovery, communication, and basic consensus concepts. However, real-world distributed systems frequently encounter node crashes, network disruptions, and dynamic topology changes. Handling these failures manually within application code is notoriously error-prone.

In this exercise, you will implement a small distributed auction using Apache ZooKeeper for coordination. The system must accept bids from multiple nodes, determine a winner, and preserve a consistent auction result across node failures.

You may use any programming language with a ZooKeeper client library.

Requirements

Run at least five application nodes on five separate machines (e.g., virtual machines, Vagrant VMs, or EC2 instances).

Use ZooKeeper to coordinate the nodes and store the authoritative auction state.

- A single auction with a single item per auction is sufficient.
- A standalone ZooKeeper server is sufficient; a ZooKeeper ensemble is not required.
- A command-line interface is sufficient. No graphical interface or web server is required.
- Nodes may communicate exclusively through ZooKeeper.
- The implementation must also work when nodes communicate over a network, rather than relying on shared memory or local files.

Task 1 – Distributed Auction

Implement a distributed auction that allows participants to submit bids through different application nodes.

Your application must support:

1. **Place a bid:** A participant submits an identifier and a bid amount. A bid is accepted only if it exceeds the current highest bid.
2. **Query the auction:** Display the highest accepted bid and the corresponding participant.
3. **Close the auction:** Determine the winner based on the highest accepted bid. An auction without any accepted bids has no winner.

All nodes must use ZooKeeper to coordinate the auction state.

You can choose how to represent the auction state in ZooKeeper and how nodes access and update it.

Experiment: Start five application nodes on different machines. Submit bids through both nodes and verify that they observe the same auction state after the updates have completed.

Task 2 – Concurrent Bids

Participants may submit bids simultaneously through different nodes.

Extend your implementation to ensure that concurrent operations do not lead to inconsistent results.

Your system must guarantee that:

- Concurrent bids cannot overwrite a higher accepted bid with a lower one.
- The auction can only be closed once, with a single final result.
- Bids cannot be accepted after the auction has been closed.
- A bid submitted concurrently with auction closure is either committed before closure or rejected.

Use suitable ZooKeeper mechanisms to enforce these properties. You may choose the coordination approach, but a solution based only on local application locks is not sufficient.

Experiment:

1. Submit two competing bids simultaneously through different nodes.
2. Attempt to submit a bid while another node closes the auction.
3. Verify that the resulting state satisfies the requirements.

Explain how your implementation prevents conflicting updates.

Task 3 – Crash Recovery and Stale State

An application node may crash or become disconnected while other nodes continue processing bids.

Extend your implementation so that application-node failures do not cause inconsistent auction outcomes.

Your system must ensure that:

- Accepted auction state is not lost when an application node crashes.
- A restarted node retrieves the current auction state from ZooKeeper.
- A node with outdated local information cannot overwrite a newer state or declare a conflicting winner.
- A node cannot successfully modify the auction while it cannot coordinate with ZooKeeper.

Experiment:

1. Start Nodes A and B and submit a bid through Node A.
2. Terminate Node A.
3. Submit a higher bid through Node B and close the auction.
4. Restart Node A.
5. Verify that Node A recognizes the finalized result and cannot change it.
6. Evaluate your implementation across multiple auctions with randomly injected faults (crashes, disconnects, etc.).

Describe how your implementation prevents a recovered node from acting on outdated state.

Questions

- How is the auction state represented in ZooKeeper? Which parts must be persistent?
- Which ZooKeeper mechanisms prevent conflicting updates from concurrent nodes?
- What happens if two nodes try to close the auction simultaneously?
- How does a restarted node obtain the correct auction state?
- What happens if an application node cannot reach ZooKeeper? Why might rejecting operations be preferable to continuing with locally cached state?

Learning objectives

After completing this exercise, you should be able to:

- explain the role of coordination services in distributed systems;
- differentiate between application-level fault tolerance and infrastructure coordination primitives;
- use ephemeral nodes for automatic failure detection;
- implement event-driven updates using ZooKeeper watches in Kotlin.

Submission

Submit a PDF file with your solution and your code. You can optionally use the markdown template provided in OLAT (`Submission_Template.md`) and convert it to PDF.

Your report should address all tasks and questions specified in the exercise. Depending on the exercise, this may include:

- theoretical questions and explanations;
- a description of your implementation;
- source code snippet;
- experimental results, observations, or screenshots;
- answers to the discussion questions.

Focus your report on the **distributed-systems concepts, your design decisions, and your observations**. Keep the report concise and avoid documenting routine installation steps unless they are relevant to the exercise.

The submission should follow the following structure and naming convention:

```
|-- exc0X_report_csXXXXXX.pdf
`-- exc0X_source_csXXXXXX.zip
    |-- README.md
    |-- sample_file_a.py
    |-- sample_file_b.py
    `-- sample_file_c.py
```

Deadline

19.11.2026 12:00h

Presentation

20.11.2026